



Wrocław University
of Science and Technology

Raport - zadanie 2

Problem komiwojażera (TSP) - Branch and Bound z wariantami BFS, DFS i LC

Imię i nazwisko: Robert Tworek

Numer indeksu: 276272

Przedmiot: Projektowanie Efektywnych Algorytmów

W raporcie przedstawiono implementację dokładnego algorytmu Branch and Bound dla TSP oraz porównanie trzech strategii przeglądania przestrzeni rozwiązań: BFS, DFS i LC.

1. Treść zadania, cel i hipotezy

W zadaniu 2 należało zaimplementować algorytm oparty o metodę podziału i ograniczeń (Branch and Bound) oraz porównać trzy warianty przeglądania przestrzeni rozwiązań: BFS, DFS i LC.

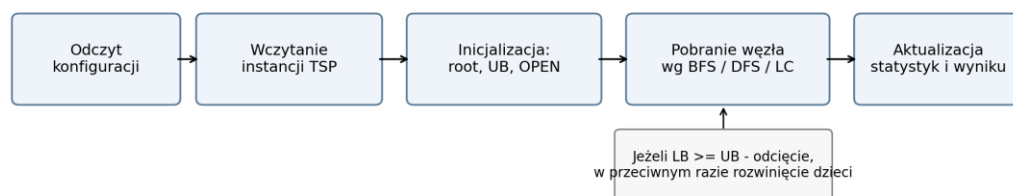
Celem pracy było zbadanie wpływu strategii wyboru kolejnego wężła na czas działania, liczbę rozwijanych stanów oraz pamięciochłonność mierzoną maksymalnym rozmiarem struktury OPEN. Przyjąłem hipotezy, że BFS będzie wariantem najsłabiej skalowalnym, DFS najlepszym pamięciowo, a LC najskuteczniejszym w ograniczaniu liczby analizowanych węzłów.

2. Metoda, implementacja, dane i konfiguracja

Program zaimplementowałem w języku Python jako samodzielną aplikację sterowaną plikiem konfiguracyjnym. Każdy węzeł drzewa przechowuje częściową ścieżkę, zbiór odwiedzonych miast, bieżący koszt, poziom drzewa oraz wartość ograniczenia dolnego LB. Ograniczenie dolne wyznaczałem jako koszt dotychczasowej ścieżki powiększony o optymistyczne oszacowanie minimalnych kosztów wyjścia z bieżącego miasta i z miast nieodwiedzonych. Algorytm utrzymuje również najlepsze dotąd rozwiązanie UB; jeżeli $LB \geq UB$, gałąź jest odcinana.

Procedura badawcza polegała na uruchomieniu każdego z trzech wariantów algorytmu dla wszystkich 18 instancji testowych oraz zapisaniu czasu działania, liczby rozwiniętych i wygenerowanych węzłów, liczby odcięć, maksymalnego rozmiaru OPEN i informacji o timeoutach.

BFS wykorzystuje kolejkę FIFO, DFS stos LIFO, a LC kolejkę priorytetową uporządkowaną według najmniejszego LB. Zestaw badawczy obejmował 18 wygenerowanych instancji metrycznych i symetrycznych: po 3 instancje dla $n = 6, 7, 8, 9, 10$ i 11 . W konfiguracji ustawiono $start_city = 0$, $time_limit_s = 5$ oraz $use_nn_ub = false$.



Rysunek 1. Schemat działania programu wykorzystanego w zadaniu 2.

Tabela 1. Najważniejsze pola pliku konfiguracyjnego wraz z przykładowymi wartościami.

Parametr	Znaczenie	Przykład
instances_dir / pattern	katalog i wzorec plików wejściowych	instances_metric / *.txt
modes	wybór wariantów przeszukiwania	bfs, dfs, lc
start_city	miasto startowe	0
time_limit_s	limit czasu [s] dla pojedynczego uruchomienia	5

use_nn_ub	inicjalizacja UB heurystyką	false
summary_csv	zbioreczy plik wynikowy	batch_results_summary.csv

3. Wyniki badań dla BFS, DFS i LC

Wraz ze wzrostem rozmiaru instancji różnice między wariantami stawały się coraz bardziej widoczne. Największe kontrasty dotyczyły czasu działania, liczby rozwijanych węzłów oraz maksymalnego rozmiaru OPEN.

Tabela 2. Średnie wyniki dla wszystkich badanych uruchomień (18 uruchomień na tryb).

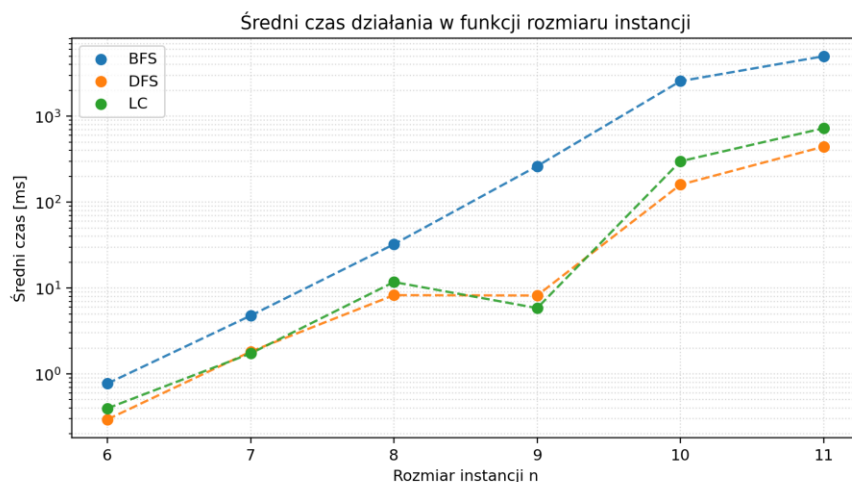
Tryb	Runs	Śr. czas [ms]	Śr. expanded	Śr. generated	Śr. pruned	Śr. max OPEN	Timeouty	Potw. optimum
BFS	18	1 310.777	190 027.9	442 286.5	68 167.0	252 271.2	3	15
DFS	18	103.609	10 767.4	35 485.4	24 717.9	26.8	0	18
LC	18	174.091	8 080.7	27 041.8	18 961.1	18 962.1	0	18

BFS był wyraźnie najdroższy obliczeniowo, DFS najlepszy czasowo i pamięciowo, a LC rozwinął najmniej węzłów.

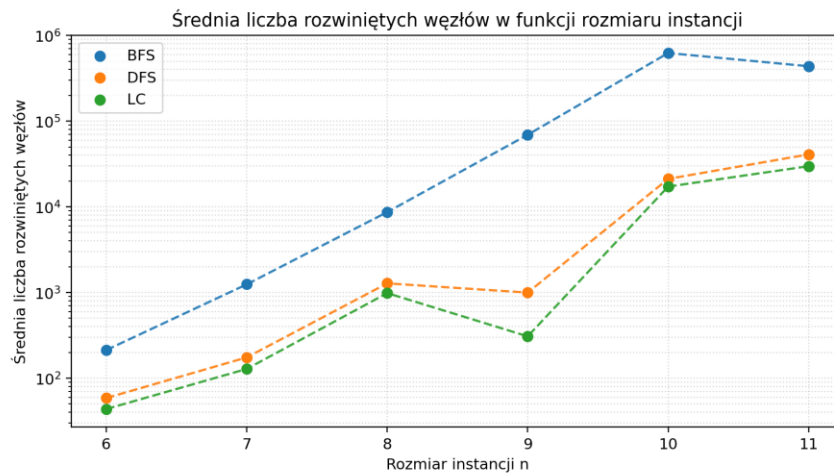
Tabela 3. Średni czas działania i liczba timeoutów w zależności od rozmiaru instancji.

n	BFS [ms]	DFS [ms]	LC [ms]	Timeouty (BFS/DFS/LC)
6	0.772	0.295	0.397	0/0/0
7	4.771	1.817	1.730	0/0/0
8	32.294	8.260	11.737	0/0/0
9	262.727	8.182	5.838	0/0/0
10	2 564.097	159.749	299.208	0/0/0
11	5 000.003	443.354	725.638	3/0/0

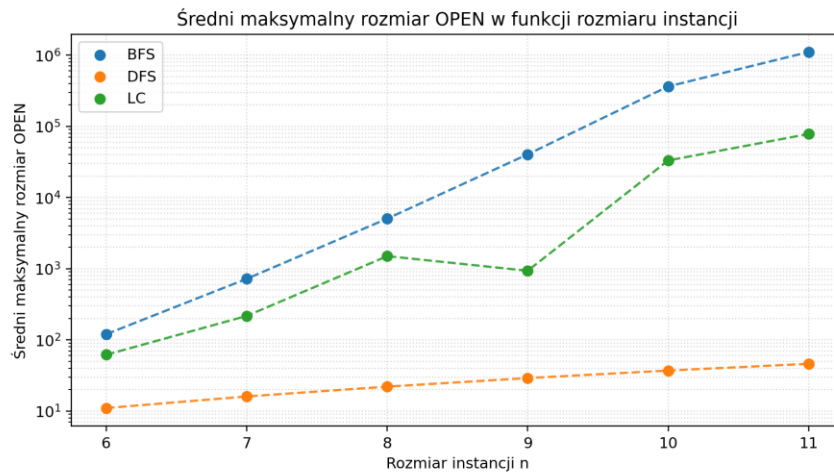
Najbardziej krytyczny punkt pojawił się dla $n = 11$: BFS przekroczył limit 5 s dla wszystkich trzech instancji, podczas gdy DFS i LC ukończyły wszystkie uruchomienia.



Rysunek 2. Średni czas działania w funkcji rozmiaru instancji; punkty oznaczają średnie pomiary, a linie przerywane trend zmian. Oś Y przedstawiono w skali logarytmicznej.



Rysunek 3. Średnia liczba rozwiniętych węzłów w funkcji rozmiaru instancji; punkty oznaczają średnie pomiary, a linie przerywane trend zmian. Oś Y przedstawiono w skali logarytmicznej.



Rysunek 4. Średni maksymalny rozmiar OPEN w funkcji rozmiaru instancji; punkty oznaczają średnie pomiary, a linie przerywane trend zmian. Oś Y przedstawiono w skali logarytmicznej.

4. Analiza wyników, odpowiedzi na pytania i wnioski

BFS okazał się wariantem zdecydowanie najmniej praktycznym. Wraz ze wzrostem n liczba aktywnych węzłów rosła bardzo szybko, co przełożyło się na wysoki koszt pamięciowy i czasowy. Dla $n = 11$ średni maksymalny rozmiar OPEN przekroczył 1 04 547, a wszystkie trzy uruchomienia zakończyły się timeoutem.

DFS był najlepszym wariantem pod względem czasu działania i zużycia pamięci. Średni czas dla całego zestawu wyniósł 103.609 ms, średni maksymalny rozmiar OPEN 26.8, a liczba timeoutów była równa 0. Oznacza to, że w badanym zakresie DFS był najstabilniejszą i najbardziej praktyczną strategią przeszukiwania.

LC uzyskał najmniejszą średnią liczbę rozwiniętych węzłów: 8080.7 wobec 10 767.4 dla DFS i 190 027.9 dla BFS. Wariant least-cost najskuteczniej ograniczał więc liczbę analizowanych stanów, ale jednocześnie wymagał wyraźnie większego OPEN niż DFS i nie był od niego szybszy.

Na podstawie badań stwierdzam, że BFS nadaje się głównie do bardzo małych instancji i celów poglądowych. DFS jest najbardziej praktyczny, gdy priorytetem są czas i pamięć, natomiast LC jest korzystny wtedy, gdy ważniejsze jest jak najmocniejsze zawężanie przestrzeni przeszukiwania.

Bibliografia

- [1] J. D. C. Little, K. G. Murty, D. W. Sweeney, C. Karel, "An Algorithm for the Traveling Salesman Problem", Operations Research, 11(6), 1963.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, Wprowadzenie do algorytmów.
- [3] D. L. Applegate, R. E. Bixby, V. Chvátal, W. J. Cook, The Traveling Salesman Problem: A Computational Study, Princeton University Press, 2006.



Wrocław University
of Science and Technology
